

Next run monitor

Experimental. We are just trying this. Private note. TN10 only. Not sent. Sending this GitHub to Grok Build or to the bot means go: start the operation. The bot's runner and the bot's miners use the tunnel to keel. The runner runs only during the storm. n0 will not run.

Source: STP-KAS/tn10-locus plan/NEXT-RUN-MONITOR.md. If this PDF and that page disagree, the page wins. If the page and NEXT-STORM-PLAN.md disagree on a clock, a fee, or a question, the plan wins. If they disagree on the node, PLAN.md wins.

Clock	Local, Europe/Brussels, CEST (UTC+2)	UTC
This evening only	Friday 9 Oct 2026. No Monday. No Tuesday.	Same calendar date.
T0, start	23:00	21:00
B0 ends, first load step	23:10	21:10
End, 8 hours	Saturday 10 Oct 2026, 07:00	Saturday 10 Oct 2026, 05:00
His counter, does not move T0	23:25 through 07:35	21:25 through 05:35
Gap	Our T0 is 25 minutes before his counter.	21:00 versus 21:25.
Dropped fallback	Not Monday 12 Oct. Not Tuesday 13 Oct.	13 Oct 2026 is a Tuesday, not a Monday.
Unnamed hours	After 02:25 local	After 00:25 UTC. Storm GO names them, or ends at 00:25.

keel is the second desk kaspad. locus is the first. The desk watches the block download and leaves the tunnel closed until keel is synced. When keel is synced, the desk opens the tunnel, writes nodeB_wrpc and nodeB_grpc in the handoff, and says both addresses. That report is not the storm GO. Until those fields are real, the bot's runner and the bot's miners wait, the row says waiting, and keel adds 0. From the box, do not use 127.0.0.1. Do not invent a host. Miners pay the Grok Bot address kasptest:qzffl5xy9np46gkttuyftqnv2w04pr8g3wsp7c3vv8se3txtelx6q7c0v0ldx and mine only while keel is synced. Desk miners stay on locus until then. On the desk, keel's own sockets are Borsh ws://127.0.0.1:17310 and gRPC 127.0.0.1:16310. Those are not the box's addresses.

A required line with no reading is not measured and fails the pass. Do not invent the other side. Do not ping Kaspa Pulse. No key, no seed, and no txid in git. The mining address is the one above.

What 8 Oct left unread

The next run reads each open line. A repeat of the blank fails the pass. The three n0 lines are closed: n0 will not run, so they are not a failed pass. The next run reads keel instead: synced, lag, mempool, CPU, and mempool every second. Saturation is scored on keel for the bot and on locus for Build.

Line	8 Oct
Bot block, every 10 minutes	Not printed. Every bot cell is not measured.
Rows 18:00 and 18:10 UTC	Not read.
Usage, both sides	Not measured.
n0 synced, lag, mempool, CPU	Closed. n0 will not run. Read keel instead.
Box disk and box NTP	Not read.
Desk NTP at 18:00 and at 20:00	Not both read. The 50 ms flag was not scored.
Per minute: tx sent, node, five ids	Evening locus rounds logged no per-transaction ids. The 18:30 sample was skipped.
Indexer every 30 seconds, plus one visibility poll	One sample on the 10-minute row. No freeze by the written rule.
n0 mempool every second, fee every 10 seconds	Closed. n0 will not run. Read keel mempool every second.
Confirmation time, 1x against 1.5x	Not measured.
Order under load	Ordered stream off. Not measured.
Saturation on n0	Closed. Score keel for the bot and locus for Build.
Mining share	Not measured.
Miners-off control at the 2x load	Left for this next run.
Combined accepted tx/s	Bot side missing. Locus accept is not the combined rate.

Before T0

Stop unless this GitHub has been sent to Grok Build or to the bot, steps-utc.json is in hand, and the clock is at or after the first time in that file. That send is the storm GO. The runner runs only during the storm.

Check	Owner	Pass
keel still syncing	Desk	Watch the block download. Leave the tunnel closed. When keel is synced, open the tunnel and write nodeB_wrpc and nodeB_grpc in the handoff. Say both addresses. That report is not the storm GO. Until then the bot row says waiting.
keel synced, lag seconds, mempool, box disk	Bot	Runner only during the storm. Send only if synced and lag is at or under 300 seconds, and the disk gate passes. Otherwise sender count 0 and the row says waiting.
keel tunnel for the bot	stp	stp provides the tunnel when keel is synced. Until it is in the handoff, the runner and the miners wait.
Locus synced, UTXO index, mempool, fee, free RAM, desk disk	Build	Loopback Borsh on locus. Free RAM under 1 GB, or a dead process, stops Build.
NTP, five samples, both machines	Each side	Repeat at the end. A desk move over 50 ms flags Build's confirmation times.
Older fleet halt file	Build	First line still halt.

Check	Owner	Pass
Normal fee quote, own node	Each side	If the quote is already above 200, that side does not start.

Every 10 minutes, both sides

The desk writes the sheet. The box prints its block. Copy the block. A missing block stays not measured.

Field	Bot, keel	Build, locus
Session	up, down, or waiting	up or down
Sender count	lane runners, or 0 while waiting	Build senders
Disk	box GB	desk GB
Clock	NTP offset	w32tm, 5 samples at start and end
Usage	product counter, or not measured	product counter, or not measured
Node	synced, lag, mempool, CPU	synced, UTXO index, mempool, fee, free RAM
Rates	submitted tx/s and accepted tx/s	submitted tx/s and accepted tx/s
Combined	both sums, one line	same two sums
Pools	keel mempool	locus, plus six public names
Indexer	latest 30-second sample	same check if the box row is missing
Miners	bot miners, through the keel tunnel, or off while keel syncs	desk miners, still on locus until keel is synced

While the bot's runner is waiting, the combined accepted rate for that minute is Build's rate alone, and the row says so. keel still adds 0.

Every second and every minute

Each UTC second, per sender. Sender id, target, submitted, accepted, endpoint keel or locus, that node's mempool, submit latency. Accept time is when that side's own node reports the transaction. Per-transaction log stays on. On a sample, and on every reject: submit time and accept time, UTC, milliseconds, Z. Each UTC minute, per sender. Minute, sender id, node keel or locus, tx_sent, five tx ids spread across the minute. Not the first five. Not the word public. One combined minute line. Both submitted sums and both accepted sums. Ids stay local. Git gets the count. His accepted count is his.

His five questions

#	Question	What the next run logs
1	Accepted against submitted, and where it flattens	Per second, both sides, by fee tier. Saturation: from 60 seconds after the step start, accepted under 95% of submit for 60 consecutive seconds. Report yes or no, and the onset. The chain rule uses keel for the bot. A waiting runner is not a zero-second failure.
2	Confirmation time, median and worst, 1x against 1.5x	Sequence, submit time, accept time, fee tier. About 450 probes per tier per step. Report n, p50, p95, p99, worst, and the share over 30 seconds and over 60 seconds.
3	Indexer freeze, at what rate, and for how long	api-tn10 health every 30 seconds. Freeze: 3 bad samples, lag above 120 seconds and rising, or visibility delay above 300 seconds. One id polled each minute. The id stays local.
4	Mempool depth over time	keel mempool every second. Fee every 10 seconds. Locus mempool on the second line. Six public names on the 10-minute row.
5	Whether order holds under load	Ordered stream, 2 per second at 1x and at 1.5x. Reorder, out-of-order, stalls, and fee overtakes, as counts and percentages, with n and a 95% interval.

The other 95% rule stays separate. The Build gate is mean submit at least 95% of that side's target, with no zero second on a sender that was armed. Sender-limited means submit stayed under 95% of target. Do not call that a network ceiling.

Also on this run, already in the plan

- B0 is 10 minutes, 23:00 to 23:10 local, 21:00 to 21:10 UTC. Lane senders off. Box probes and the ordered stream stay on. Build sends nothing in B0.
- Steps stay 2x, 5x, 10x, 20x, 30x, then max. Nothing changes inside a step. Never eight box runners.
- Miners-off control at the 2x load, matched to miners on. The control is imperfect. Say that next to the pair. Our share on that step should read about 0%.
- When accepted flattens, log keel CPU, mempool-cap hits, and reject reasons. The old n0 cap does not apply to locus or to keel.
- Mining share per node, per step: blocks total, blocks ours, percent. The bot's miners use the keel tunnel. Desk miners stay on locus while keel is syncing.
- At T0, cite the plan SHA. A deviation gets a time and a reason. Raw files stay the plan's section 8 list.

After the run, private. Not in git.

Not on the sheet. Not sent with the storm GO. Two asks from the 9 Oct reply:

1. Step times from steps-utc.json, so minutes line up with B0, 2x, 5x, and the rest.
2. Our five ids per minute, so he can check on chain if and when each one landed.

He can count, from the chain, how many blocks paid the mining address per step. That is mining share from outside, next to our own number. Comparison comes to us first.

Pass

The run does not claim the combined goal unless Task 10 of MONITOR.md is complete and every open 8 Oct blank above has a reading, or a written not measured with the UTC it was due. The closed n0 lines are not a failed pass. Kasper Pulse counts the chain from outside the setup. Comparison comes to us first. This PDF was not sent.